

Experiment-03: Water Jug Problem using Python (BFS)

Aim

To develop a Python program that solves the **Water Jug Problem** using the **Breadth-First Search (BFS)** algorithm and determines the sequence of steps required to obtain the target amount of water.

Problem Statement

Two jugs are given:

- Jug A capacity = **4 liters**
- Jug B capacity = **3 liters**
- Target = **2 liters**

Allowed operations:

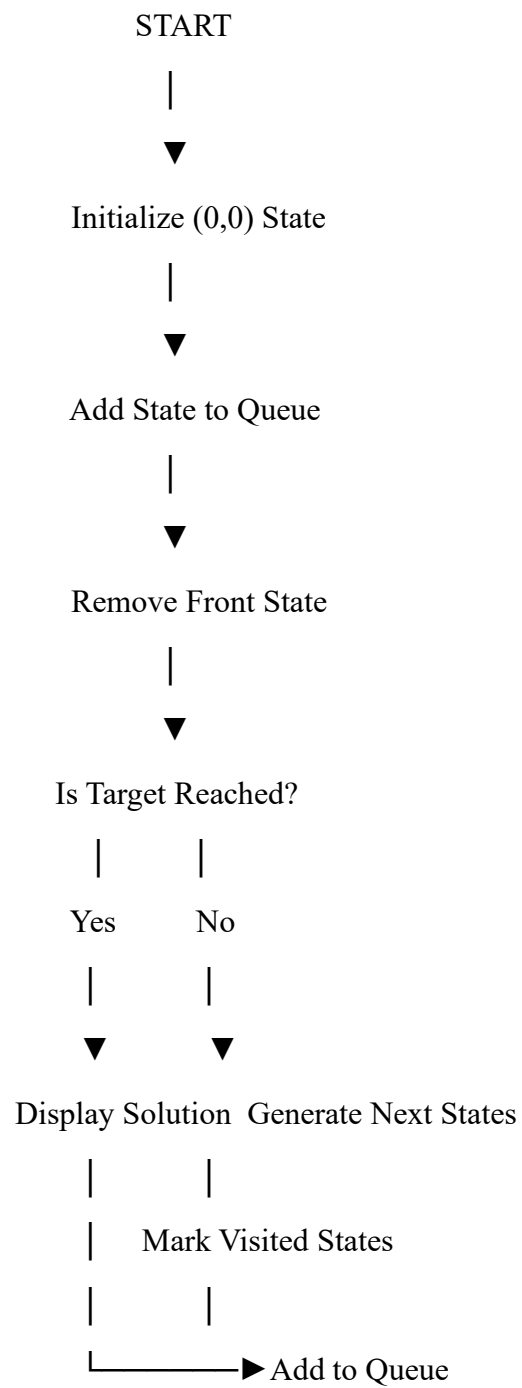
- Fill a jug completely.
 - Empty a jug completely.
 - Pour water from one jug to another until one jug becomes empty or the other becomes full.
-

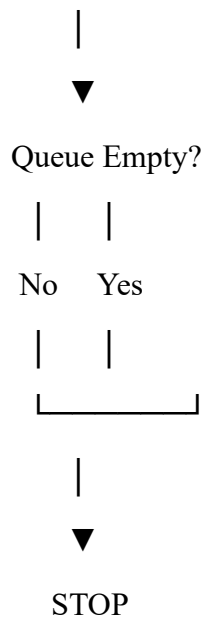
Algorithm

1. Start with both jugs empty **(0,0)**.
2. Store the initial state in a queue.
3. Mark visited states to avoid repetition.
4. Remove the first state from the queue.
5. If the target amount is reached, stop.
6. Generate all possible next states:
 - Fill Jug A
 - Fill Jug B
 - Empty Jug A
 - Empty Jug B

- Pour Jug A $\rightarrow$ Jug B
 - Pour Jug B $\rightarrow$ Jug A
7. Add unvisited states to the queue.
 8. Repeat until the target is found.
 9. Print the sequence of states.

Flowchart





Python Code

```
from collections import deque
```

```
def water_jug_bfs(capacity1, capacity2, target):
```

```
    queue = deque()
```

```
    visited = set()
```

```
    queue.append((0, 0, []))
```

```
    while queue:
```

```
        jug1, jug2, path = queue.popleft()
```

```
        if (jug1, jug2) in visited:
```

```
            continue
```

```
        visited.add((jug1, jug2))
```

```
path = path + [(jug1, jug2)]
```

```
if jug1 == target or jug2 == target:
```

```
    return path
```

```
next_states = [
```

```
    (capacity1, jug2),    # Fill Jug1
```

```
    (jug1, capacity2),    # Fill Jug2
```

```
    (0, jug2),            # Empty Jug1
```

```
    (jug1, 0),            # Empty Jug2
```

```
]
```

```
# Pour Jug1 -> Jug2
```

```
transfer = min(jug1, capacity2 - jug2)
```

```
next_states.append((jug1 - transfer, jug2 + transfer))
```

```
# Pour Jug2 -> Jug1
```

```
transfer = min(jug2, capacity1 - jug1)
```

```
next_states.append((jug1 + transfer, jug2 - transfer))
```

```
for state in next_states:
```

```
    if state not in visited:
```

```
        queue.append((state[0], state[1], path))
```

```
return None
```

```
capacity1 = 4
```

```
capacity2 = 3
```

```
target = 2
```

```
solution = water_jug_bfs(capacity1, capacity2, target)
```

```
if solution:
```

```
    print("Steps to reach the target:\n")
```

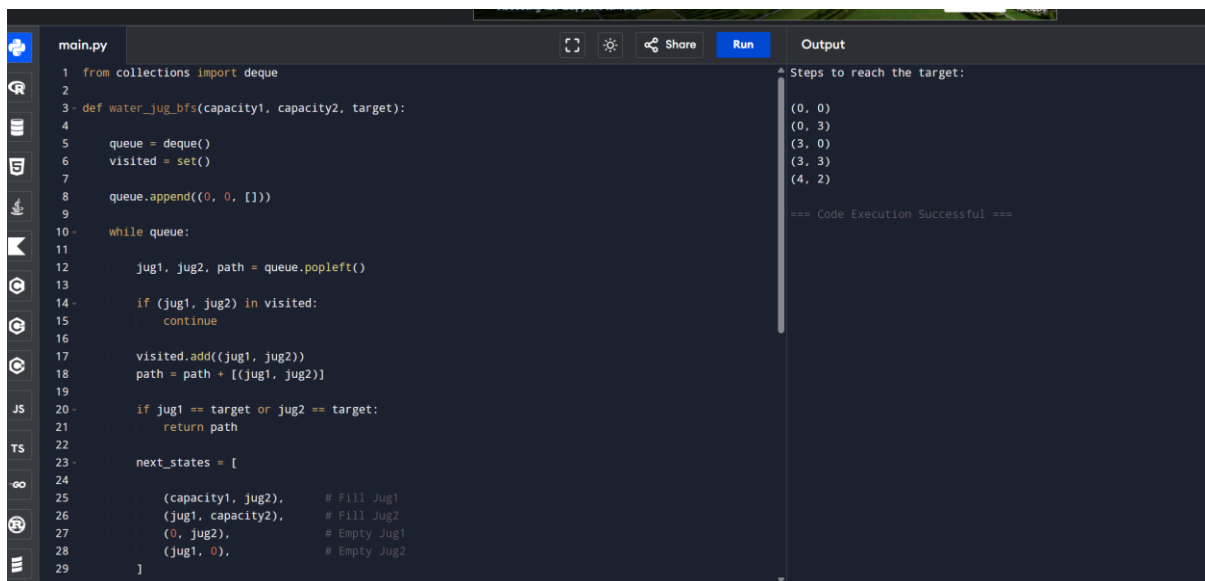
```
    for step in solution:
```

```
        print(step)
```

```
else:
```

```
    print("No solution found.")
```

Output



```
main.py
1 from collections import deque
2
3 def water_jug_bfs(capacity1, capacity2, target):
4
5     queue = deque()
6     visited = set()
7
8     queue.append((0, 0, []))
9
10    while queue:
11
12        jug1, jug2, path = queue.popleft()
13
14        if (jug1, jug2) in visited:
15            continue
16
17        visited.add((jug1, jug2))
18        path = path + [(jug1, jug2)]
19
20        if jug1 == target or jug2 == target:
21            return path
22
23        next_states = [
24
25            (capacity1, jug2), # Fill Jug1
26            (jug1, capacity2), # Fill Jug2
27            (0, jug2), # Empty Jug1
28            (jug1, 0), # Empty Jug2
29
30        ]
```

Steps to reach the target:

```
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)

=== Code Execution Successful ===
```

Result

The **Water Jug Problem** was successfully solved using the **Breadth-First Search (BFS)** algorithm. The program found the shortest sequence of operations required to obtain the target amount of water (2 liters) while avoiding repeated states.